IMPLEMENTATION, DEMONSTRATION & DEFENSE HANDBOOK

ScamShield Inside Out

How the real-time SMS scam detection system works, how the code maps to the strict pitched requirements, what can be demonstrated today, and what must be completed for full SRS compliance.

Source baseline: *ScamShield_SRS_Strict_Pitched_Features.docx*

Implementation: Android / Java / local deterministic analysis

Prepared: September 2026

How to use this guide

This document has two purposes. First, it teaches the team enough technical detail to explain the system confidently. Second, it acts as an honest verification report against FR01–FR18 and NFR01–NFR08 in the strict SRS.

> **Credibility rule:** do not describe a planned feature as already implemented. A supervisor can verify code, permissions, and runtime behavior. This guide labels each requirement as Implemented, Partial, Not implemented, or Extra.

Recommended reading order

1. Read the one-page system summary and architecture.
2. Study the real-time path until every team member can explain it without notes.
3. Learn the detector examples and limitations.
4. Review the requirement traceability tables.
5. Rehearse the demonstration and supervisor questions.

1. Executive system summary

ScamShield is an Android application that reacts to incoming SMS events, performs local rule-based text checks, classifies detected scam patterns, and warns the user through a notification. It also supports manual analysis and currently contains optional history and reporting screens.

What is technically strong

- Manifest-registered real-time SMS receiver
- Multipart SMS reconstruction
- Local analysis with no cloud AI
- Deterministic keyword/type rules
- Matched-indicator explanations
- High-priority warning notification
- Manual and shared-message analysis
- Successful debug build

What still requires completion evidence

- Physical-device SMS lifecycle tests
- Permission grant/denial device tests
- Foreground-service survival tests
- Rule calibration using labelled SMS samples
- Removal of out-of-scope database/history/reporting UI
- Measured one-second performance evidence

> **Verified build status:** `testDebugUnitTest`, `assembleDebug`, and `lintDebug` completed successfully on 3 September 2026. Ten unit tests passed, including nine detector tests for safe, suspicious, scam, regex/context, normalization, null input, false positives, known numbers, and score bounds. Real-device SMS/service testing remains required.

2. What was promised in the strict pitch

The SRS deliberately narrows the project to six core capabilities:

1. Request and clearly manage SMS and notification permissions.
2. Allow the user to enable or disable real-time protection.
3. Receive incoming SMS events while protection is enabled.
4. Analyze locally using normalization, tokens, keywords, phrases, regex, contextual rules, and weights.
5. Calculate a bounded score and classify SAFE, SUSPICIOUS, or SCAM.
6. Maintain protection in the background and alert for suspicious/scam results.

The SRS explicitly excludes OCR, VPN, cloud synchronization, community blacklists, user accounts, automatic blocking, external AI, and direct telecom/TCRA integration. These scope limits protect the project from becoming too large and give the team a precise claim to defend.

> **Important contradiction with the current UI:** the app contains history/database and guided TCRA/operator reporting features that the strict SRS calls out of scope. The screenshot/OCR option exists but only displays “unavailable.” These extras should be described as prototypes or optional extensions—not part of the pitched core.

3. Current architecture: the real incoming-SMS path

```
Settings switch → permissions → ProtectionManager → foreground ProtectionService

                    Android telephony framework
                  ↓ SMS_RECEIVED broadcast
              SmsReceiver checks ProtectionManager.isActive()
            ↓ reconstruct sender + multipart body + timestamp
                          WhitelistUtils
                        ↓ if not ignored
                  ScamDetector.detect(message, sender)
          ↓ classification + 0–100 score + category + weighted indicators
                if SUSPICIOUS/SCAM → NotificationHelper warns user
```

Step 1 — Android receives an SMS

The operating system, not ScamShield, receives the cellular message. Android sends the protected `android.provider.Telephony.SMS_RECEIVED` broadcast to registered apps that have `RECEIVE_SMS` permission. ScamShield declares both the permission and `SmsReceiver` in `AndroidManifest.xml`.

Because the receiver is manifest-declared, Android can start the app process to deliver the broadcast even when no ScamShield screen is visible. `SMS_RECEIVED_ACTION` is one of Android's exceptions to the general restrictions on manifest-declared implicit broadcasts.

Step 2 — The receiver reconstructs the message

`Telephony.Sms.Intents.getMessagesFromIntent(intent)` extracts one or more `SmsMessage` parts. The receiver takes the sender from the first part and appends every body part into one complete string. This matters because long SMS messages can arrive as multiple protocol units.

```
SmsMessage[] messages = Telephony.Sms.Intents.getMessagesFromIntent(intent);
StringBuilder body = new StringBuilder();
for (SmsMessage part : messages) {
    if (sender == null) sender = part.getDisplayOriginatingAddress();
    body.append(part.getMessageBody());
}
```

Step 3 — Service messages may be ignored

`WhitelistUtils.isServiceMessage(sender, message)` ignores configured service shortcodes and also considers alphanumeric sender IDs with service wording. This attempts to reduce false alerts from operators and banks.

> **Known risk:** the current whitelist accepts every numeric sender containing 3–5 digits as a service number. A scam sent from a short code could therefore be ignored. The declared `WHITELIST_PREFIXES` set is unused. Whitelist rules require documented, tested sources rather than broad assumptions.

Step 4 — Local deterministic analysis

`ScamDetector.detect()` checks three known phone numbers and a hard-coded keyword array. If nothing matches, it returns `isScam=false`, type `Unknown`, danger `Low`. If a match occurs, `determineScamType()` chooses a type such as healer, money request, fake phone number, prize, or bank/loan and normally assigns High Risk.

Step 5 — Side effects

Every analyzed incoming message increments the SharedPreferences scan counter. Scam results are written to the local SQLite `scams` table, then `NotificationHelper` creates or reuses a high-importance notification channel and displays a vibration alert.

> **Privacy correction needed:** `SmsReceiver` logs the full sender and message body with `Log.d`. Production builds should never log private SMS content. Remove or redact these logs before a real deployment or privacy demonstration.

4. The manual-analysis path

```
User opens Analyze
↓ paste / manual input / Android share sheet
AnalyzeFragment
↓ same ScamDetector used by receiver
ScanResultFragment
↓ verdict, danger, type, preview, indicators
optional → Details / Report screens
```

This path is valuable during a classroom demonstration because it does not require waiting for a real SMS. It proves that the UI and receiver share the same detector rather than maintaining different logic.

- **Paste:** reads the clipboard and opens the sender/message form.
- **Manual:** opens the form directly.
- **Share:** Android sends selected text to `MainActivity`, which routes it to Analyze.
- **Read SMS Automatically:** requests `RECEIVE_SMS` and `READ_SMS`.
- **Screenshot:** intentionally shows that OCR is unavailable; the strict SRS excludes OCR.

A clean message is not stored. A detected scam is stored and triggers a notification even when manually analyzed.

5. Detector logic inside out

Current decision process

```
INPUT: message, sender

1. Normalize nulls, Unicode, case, quotes and whitespace.
2. Tokenize the normalized message.
3. Evaluate every precompiled DetectionRule once.
4. Add each matched rule's weight and explanation.
5. Apply explicit contextual-combination bonuses.
6. Bound score to 0..100.
7. Classify: 0–29 SAFE, 30–59 SUSPICIOUS, 60–100 SCAM.
8. Return class, score, strongest category, indicators and tokens.
```

The outcome is deterministic: identical input produces identical output while the arrays and code remain unchanged. It runs entirely on-device and makes no network request. The manifest does not request `INTERNET`, which strongly supports the “no external AI API” claim.

Examples the team should understand

Example	Matched logic	Current result
“Nitumie pesa kwa AirtelMoney sasa”	Money request + urgency	45 / SUSPICIOUS
“Hongera, umeshinda zawadi. Lipa shilingi 5000 sasa.”	Prize + payment + urgency + context	95 / SCAM
“Akaunti yako ya benki imefungwa”	Account threat + institution mention	40 / SUSPICIOUS
“Mtaalam wa tiba atakurudishia mali”	Healer/supernatural promise	45 / SUSPICIOUS
“Nimefika salama nyumbani”	No configured term	Not scam / Low

What “normalization” means in the SRS

The implemented normalization stage now:

1. Handle null input as an empty string.
2. Apply Unicode normalization.
3. Convert to a stable locale-independent lowercase.
4. Normalize spacing and punctuation.
5. Optionally normalize Tanzanian phone-number forms.
6. Retain both normalized text and tokens for rule evaluation.

What weighted scoring should look like

The implementation now uses rules with IDs, compiled patterns, weights, explanations, categories, and sender/message scope. Its design is:

Rule	Condition	Example weight	Explanation
R01	Known confirmed scam number	70	Sender/body includes confirmed number
R02	Credential request: PIN/password/OTP	35	Requests confidential authentication data
R03	Urgency phrase	15	Pressures immediate action
R04	Money request	30	Requests transfer or payment
R05	URL regex	20	Contains a link
R06	Prize wording	30	Claims a prize or unexpected reward

```
score = min(100, sum(unique matched rule weights))

0–29   → SAFE
30–59  → SUSPICIOUS
60–100 → SCAM
```

> **Important:** the 30/60 thresholds are now implemented, but the SRS did not prescribe their numerical values. The team must calibrate and justify them against a labeled Tanzanian SMS dataset.

Current detector limitations

- The initial weights and 30/60 thresholds require dataset calibration.
- Rule coverage is deliberately small and will miss unfamiliar wording.
- Obfuscation such as inserted punctuation or unusual spelling needs dedicated normalization/rules.
- The whitelist still requires trustworthy sources and false-negative tests.
- Indicators are not stored with the optional history, so reopening a saved detection loses explanations.
- The detector has nine focused unit tests, but receiver, permission and service behavior still require device tests.

6. Strict functional-requirement traceability

ID	Status	Evidence and honest interpretation
FR01	**IMPLEMENTED**	The Settings protection switch requests essential permissions before activation.
FR02	**IMPLEMENTED**	`POST_NOTIFICATIONS` is declared and requested on Android 13+.
FR03	**IMPLEMENTED**	Settings displays active, inactive, or permission-required state and explains denial.
FR04	**IMPLEMENTED**	Protection cannot be enabled without SMS and notification permissions; revocation disables it.
FR05	**IMPLEMENTED**	A persistent user-controlled protection switch starts/stops protection.
FR06	**IMPLEMENTED IN CODE**	The manifest receiver handles SMS only when `ProtectionManager.isActive()` is true; device verification remains.
FR07	**IMPLEMENTED**	Sender, complete multipart body, and the SMS timestamp are extracted.
FR08	**IMPLEMENTED**	NFKC Unicode normalization, locale-stable lowercase, quote normalization, trimming, and whitespace collapse are applied.
FR09	**IMPLEMENTED**	The normalized message is tokenized and tokens are retained in the result.
FR10	**IMPLEMENTED**	Local keyword arrays are searched case-insensitively.
FR11	**IMPLEMENTED**	Phrase-distance regex rules and three explicit contextual combinations are evaluated.
FR12	**IMPLEMENTED**	Precompiled Java `Pattern` rules cover links, phone numbers, credentials, payments, prizes, threats, and other structures.
FR13	**IMPLEMENTED**	Each `DetectionRule` has an ID, pattern, weight, explanation, category, and scope.
FR14	**IMPLEMENTED**	Unique matched rule/context weights are summed and bounded to 0–100.
FR15	**IMPLEMENTED**	0–29 is SAFE, 30–59 SUSPICIOUS, and 60–100 SCAM; classification and score are displayed.
FR16	**IMPLEMENTED**	The immediate result identifies every matched rule ID, contribution, and explanation. History persistence remains out of strict scope.
FR17	**IMPLEMENTED**	Suspicious and scam results receive distinct warning titles; safe results avoid intrusive notification.
FR18	**IMPLEMENTED IN CODE**	User activation starts a visible special-use foreground `ProtectionService`; lifecycle requires physical-device verification.

Bottom line

The strict core is now implemented in code and covered by detector tests. Final acceptance still depends on real-device permission, incoming-SMS, notification, and foreground-service lifecycle tests, followed by rule calibration and removal of optional scope.

7. Non-functional-requirement traceability

ID	Status	Assessment
NFR01 Performance	**PLAUSIBLE, UNPROVEN**	Local loops are small and likely fast, but no benchmark proves analysis normally completes within one second.
NFR02 Reliability	**PARTIAL**	Null input and core rule edge cases are tested; service/receiver continuity still needs device testing.
NFR03 Privacy	**PARTIAL**	Core analysis is local, no external AI and no Internet permission. Full SMS text is logged and scam content is stored in SQLite.
NFR04 Usability	**IMPLEMENTED**	Protection/permission state, three verdicts, score, risk, and matched indicators are visible in clear language.
NFR05 Availability	**IMPLEMENTED; VERIFY**	A sticky foreground service and manifest receiver support background operation; physical-device lifecycle testing remains.
NFR06 Maintainability	**IMPLEMENTED**	`DetectionRule`, detector, protection manager, service, notifications, and UI responsibilities are separated.
NFR07 Efficiency	**IMPLEMENTED**	Reception is event-driven; the app does not poll continuously.
NFR08 Explainability	**IMPLEMENTED**	Every indicator includes its rule/context ID, point contribution, and explanation.

8. The background-service question

This is likely to be the supervisor's hardest technical question. The implementation now deliberately uses both components: a visible foreground service represents the user's enabled protection state, while the event-driven receiver performs the short SMS analysis.

What is implemented today

The manifest receiver wakes for an SMS, while a user-started foreground `ProtectionService` makes the enabled state visible through an ongoing notification. The receiver checks the stored permission-gated state before reconstruction or analysis, so disabling protection stops monitoring behavior.

Why a permanent service is not automatically better

Modern Android restricts background services. Foreground services must be user-visible through an ongoing notification and have additional start/type rules. A permanent polling service would also conflict with NFR07's

event-driven efficiency goal.

The implemented decision

> **Strict FR18 is retained:** enabling the Settings switch after permission approval stores the state and starts a special-use foreground `ProtectionService` with an ongoing notification. `SmsReceiver` still performs event-driven reception and immediately returns when protection is disabled. The service never polls the inbox.

Do not claim that the AppCompat locale metadata service fulfills FR18. It is disabled metadata infrastructure for language persistence and is unrelated to SMS protection.

9. Storage and strict scope

The pitched core does not require a database. The current implementation nevertheless uses:

- **SharedPreferences:** onboarding state, theme, and total scanned count.
- **SQLite:** detected scam sender, message, type, date/time, and reported flag.

This enables dashboard statistics and History, but it expands privacy responsibilities and contradicts the earlier decision not to involve a database. The SRS's ERD can be treated as a logical analysis model rather than proof that persistent tables are required.

> **Strict-pitch recommendation:** remove or disable History/report storage for the assessment build, keep only a small SharedPreferences protection Boolean, and pass each `DetectionResult` in memory to the notification/result UI. If history is retained, formally revise scope and privacy requirements.

10. Implementation progress and remaining evidence

Priority	Change	Requirements closed	Acceptance evidence
DONE	Protection switch, stored permission-gated state, and receiver ON/OFF gate.	FR03–FR06	Code and build verified; device acceptance test remains.
DONE	Rule objects with IDs, compiled regex, weights, explanations, normalization and tokens.	FR08–FR13, NFR06	Nine focused detector tests pass.
DONE	Bounded 0–100 score and three visible classifications.	FR14–FR17, NFR08	Safe, suspicious, scam, combination and score-bound tests pass.
DONE	User-started special-use foreground service with ongoing notification.	FR18, NFR05	Build/lint pass; lifecycle device tests remain.
DONE	Removed unnecessary READ_SMS permission and full SMS Logcat output.	FR01, NFR03	Static audit confirms neither remains.

P1	Decide whether to eliminate SQLite history and optional reporting.	NFR03 / scope	Documented retention decision or strict no-database build.
P1	Add detector, receiver, permission, and UI tests plus a timed benchmark.	NFR01–02	Repeatable test report, not only screenshots.
P2	Hide out-of-scope OCR/report/history UI in the strict assessment build.	Scope alignment	Every visible feature has a matching requirement.

11. Demonstration plan for the supervisor

Preparation

1. Use a physical Android phone with an active SIM for the real-time portion.
2. Install a known debug APK built from the demonstrated commit.
3. Clear app data so onboarding and permission flow are visible.
4. Disable sensitive message previews on the projector if personal SMS may arrive.
5. Prepare a second phone to send controlled examples.
6. Put the phone in airplane mode with cellular re-enabled if you want to demonstrate no Internet dependency.
7. Keep Android Studio Logcat available, but remove private SMS body logging before the formal demo.

Eight-minute demo script

Time	Action	What it proves
0:00	State the strict scope: local, deterministic, event-driven, no AI/cloud.	Clear project boundary.
0:45	Show manifest permissions and receiver declaration.	Android integration is real.
1:30	Launch after clearing data and handle permissions.	Permission management.
2:15	Manually analyze a safe message.	Safe path and shared detector.
3:00	Manually analyze a prize/money scam; show indicators.	Rule matching and explanation.
4:00	Close/minimize ScamShield. Send a controlled scam SMS from phone two.	Real-time receiver operates without visible UI.
5:15	Show the notification and explain receiver → detector → alert.	End-to-end result.
6:15	Show a safe incoming SMS causes no intrusive alert.	FR17 behavior.
7:00	Present traceability honestly and state the completion plan.	Engineering credibility.

Controlled test messages

Purpose	Message	Expected current outcome
Safe	Nimefika salama nyumbani. Tutaonana kesho.	Low / no scam alert
Suspicious money request	Nitumie pesa kwa AirtelMoney sasa.	45 / SUSPICIOUS
Scam prize/payment	Hongera umeshinda zawadi. Lipa shilingi 5000 sasa.	95 / SCAM

Credential/link scam	Thibitisha OTP yako kupitia https://fake.example	75 / SCAM
Known number	Wasiliana na 0614159000 kwa maelezo.	Fake Phone Number / High
False-positive probe	Nimetuma pesa ya kodi ya nyumba.	0 / SAFE because transfer wording is not a standalone token/request

> **Keep the false-positive test in the demonstration.** It shows why the implemented detector combines weighted signals and context instead of treating every money-related word as a scam. Continue calibrating the rules with representative English and Swahili messages before calling the detector production-ready.

12. Evidence-oriented test plan

Test group	Minimum cases	Pass condition
Permissions	Grant; deny; deny permanently; notification denied; revoke later	Protection state is never misleading and app does not crash.
Protection lifecycle	ON foreground; ON minimized; OFF; restart; reboot if required	Receiver behavior exactly follows stored state.
SMS extraction	Single part; multipart; null/empty part; alphanumeric sender; phone sender	Correct sender/body/time without crash.
Normalization	Uppercase; extra spaces; punctuation; Unicode; Swahili variations	Equivalent wording produces equivalent normalized input.
Rules	Each keyword/phrase/regex; overlaps; duplicates; context combination	Expected unique rule IDs and weights.
Thresholds	Just below/at/above every boundary	Correct SAFE/SUSPICIOUS/SCAM class.
False positives	Normal bank, rent, prize, and money conversations	Benign context stays below scam threshold.
Alerts	Safe; suspicious; scam; notifications disabled	No intrusive safe alert; correct warning severity otherwise.
Performance	Short, long, multipart, worst-case matching	Measured processing normally under SRS one-second target.
Privacy	Inspect Logcat, network, database/files	No external transmission or sensitive production logs.

Automate detector tests as local JUnit tests because the detector is plain Java. Use instrumentation/device tests only for Android permissions, broadcasts, notifications, and lifecycle behavior.

13. Questions the supervisor may ask

“Where is the AI model?”

There is none, by design. The SRS commits to deterministic string analysis. The advantage is offline operation, explainability, repeatability, low latency, and no message upload. The tradeoff is that rules require careful calibration and will not understand language as broadly as a trained model.

“How does it work when the app is closed?”

Android delivers the protected SMS broadcast to the manifest receiver and can start the app process. The receiver performs short local work and posts a notification. Be precise: this is implemented through `BroadcastReceiver`, not through the background service promised by FR18.

“Can every app read SMS?”

No. The user must grant the dangerous SMS permission, Android enforces it, and distribution policies may impose additional restrictions. ScamShield receives `SMS_RECEIVED`; it is not the default SMS application.

“How do you calculate the score?”

The detector sums matched rule and contextual weights, bounds the result to 100, and exposes every contributing rule ID and point value. The result screen displays the score and classification.

“What makes SUSPICIOUS different from SCAM?”

Suspicious is the implemented 30–59 range with suggestive but incomplete evidence; Scam is the implemented 60–100 range. SAFE covers 0–29.

“Why no database?”

The pitched detection path is transient and needs no database: receive → analyze → alert. Only the protection Boolean needs small local settings storage. The current SQLite database exists for an out-of-scope history/reporting extension and should be removed from the strict build or formally added to revised requirements.

“Why not continuously scan the inbox?”

Event-driven reception is faster and more battery-efficient. Android announces a new SMS, the receiver reacts once, and no polling loop runs. This directly supports NFR07.

“What if the message contains the word ‘pesa’ but is harmless?”

The current weighted contextual rules reduce that risk: ordinary “Nimetuma pesa ya kodi” is covered by a safe unit test, while combinations such as an explicit money request plus urgency accumulate a higher score. More labeled examples are still needed for calibration.

“Does tapping Report send automatically?”

No. The current optional report flow opens the system SMS app or share sheet with prepared text. The user remains in control. It should not be described as direct TCRA API integration.

14. Team explanation responsibilities

Role	Must be able to explain
Presenter 1 — Problem/scope	Problem, objectives, why deterministic/offline, excluded features.
Presenter 2 — Android integration	Permissions, manifest receiver, multipart extraction, notification.
Presenter 3 — Detection	Normalization target, keywords/phrases/regex, weights, thresholds, explanations.
Presenter 4 — UI/demo	Manual path, real SMS path, results, safe behavior.
Presenter 5 — Quality/privacy	NFRs, tests, performance measurement, local processing, no sensitive logs.
Presenter 6 — Traceability/roadmap	FR status, gaps, background-service decision, completion priorities.

Every member should still understand the central chain: **permission → Android broadcast → receiver → normalization/rules → score/classification → explanation/alert**.

15. Code map

Component	Responsibility
`AndroidManifest.xml`	Permissions, activities, SMS receiver, intent filters.
`MainActivity.java`	Permission request, navigation host, shared-text entry.
`SmsReceiver.java`	Receive event, rebuild multipart SMS, filter, detect, store, notify.
`WhitelistUtils.java`	Service sender/shortcode filtering.
`DetectionRule.java`	Configurable compiled pattern, ID, weight, explanation, category, and scope.
`ScamDetector.java`	Normalization, tokenization, rule evaluation, contexts, scoring, classification, and evidence.
`ProtectionManager.java`	Permission-gated stored protection state and service lifecycle.
`ProtectionService.java`	Visible foreground protection state with ongoing notification.
`NotificationHelper.java`	Notification channel and visible warning.
`AnalyzeFragment.java`	Manual, clipboard, shared-message input and detector invocation.
`ScanResultFragment.java`	Verdict presentation and explanation list.
`AppPrefs.java`	Small persistent preferences and scanned counter.
`DatabaseHelper.java`	Out-of-scope SQLite history/report state.

`HomeFragment.java`	Dashboard counts and recent saved detections.

16. Final presentation statement

> **Use this wording:**
> “ScamShield already demonstrates the feasibility of real-time Android SMS reception, local deterministic detection, explainable indicators, and user notification. Our audit also identifies the exact work needed to satisfy the strict SRS fully: protection-state control, weighted regex scoring with three thresholds, test evidence, and a documented decision about the background component. We can show both the working end-to-end prototype and a requirement-by-requirement completion plan.”

This statement is stronger than claiming everything is finished. It proves the team understands the system, the Android platform, its current code, and the engineering gap between a working prototype and a verified deliverable.

Official technical references

- Android Telephony.Sms.Intents API
- Android broadcasts overview
- Android implicit broadcast exceptions
- Android services overview
- Foreground service background-start restrictions
- Android notification runtime permission

Assessment based on the supplied strict SRS and the ScamShield source tree as inspected on 3 September 2026. Statuses describe the inspected revision; they should be updated whenever implementation changes.